GHOSTWIRE

Threat intelligence, told as narrative.

#ISSUE 003 OCTOBER 2026

The Proxy in the File Manager

How a mainstream Android file manager shipped the Popa residential-proxy SDK

// INTELLIGENCE FILE — GW-003-INF //

SUBJECT:	**io.nn.alpha (“Alpha”) residential-proxy SDK**
CARRIER:	**X-Plore File Manager (com.lonelycatgames.Xplore)**
FAMILY:	**Popa family (NetNut link cited, disputed)**
SAMPLE:	**libnativesdk.so — arm64, X-Plore 4.43.10**
PRESENT:	**native lib from 4.40.01 (Oct 2024) to 4.43.10; gone by 4.44.00**
STATUS:	**NetNut domains FBI-seized Jul 2026 (cited)**

8	7	~9	10M+
SDK DOMAINS	C2 COMMANDS	MO. RESIDENT	PLAY INSTALLS

For roughly nine months, X-Plore, a mainstream Android file manager (10 million-plus Play installs), shipped an embedded residential-proxy module that enrolls the device as an exit node for third-party traffic. It is the “Alpha” variant of the Popa family; Qurium and XLab named the variant, and Synthient and Qurium link the family to the proxy provider NetNut, which NetNut disputes. The variant and its domains were already public, as is X-Plore's publisher ID (xp_gms, in Synthient's table). New here: the affected build range, and a native sample from October 2024, a lower bound roughly 16 months before Synthient's earliest date for this generation.

The story: who / what / where / when / why

X-Plore is the kind of app that sits quietly on millions of phones for years. For about nine months of its release history, some of those installs also carried a second, unadvertised job: lending out the phone's internet connection.

Inside the app was a module, `io.nn.alpha`, that on command would turn the device into a **residential proxy**, a relay that routes someone else's web traffic out through the user's home IP address. Residential proxies are sold commercially because traffic from a real consumer IP is far harder for websites to block than traffic from a data center. The buyer gets a clean-looking exit point. The person who owns the phone gets the data and battery cost, with no clear disclosure in the app's store listing.

The proxyware itself is not new. Qurium and XLab named the "Alpha" variant and its relay domains, and Synthient documented the wider Popa family, which Synthient and Qurium link to the residential-proxy provider NetNut (parent company Alarum Technologies, publicly traded). NetNut rejects that characterization, and we do not independently confirm it. Synthient's published publisher table already lists X-Plore (`com.lonelycatgames.Xplore`, publisher `xp_gms`), a mainstream file manager among mostly streaming and TV-box apps, but without dates or affected versions. This report pins down which builds carried it and when. The native library we pulled apart is present in X-Plore builds from **October 2024** (the package first appears a build earlier, in September 2024), which is roughly 16 months before the earliest sample Synthient dates to this native-code generation. It still carries a leftover `NEUPOP_LOCK` constant under its "Alpha" paint, the tell that ties it to the lineage.

It also does not simply switch on. It first contacts a server and starts only if that server returns `YES`, sending along enough about the device for the server to decide. Whether the server ever says no, and on what basis, is something we cannot see from the client.

Open question: this analysis cannot determine how the module came to be in the app, whether first-party code, a bundled third-party SDK, or a build-chain issue. We make no claim about the developer's knowledge or intent. We disclosed the findings to Lonely Cat Games and to Google Play in October 2026.

Why you should care

If your phone is used as an exit node, **your IP address gets attached to strangers' traffic.** That can mean ad fraud, credential stuffing, scraping, or worse, all appearing to originate from your home connection. It would consume mobile data and battery, and it can land your IP on block lists. The activation gate also means you cannot tell from the outside whether your specific device was ever switched on. We did not review the app's in-app consent UI, privacy policy or Play Data Safety entry, so we make no claim about what disclosure, if any, the app presents; the module does run a foreground service, which Android requires to show a persistent notification.

How else this could be leveraged

[working hypothesis, no evidence of use] Two design choices here reach past reselling bandwidth, and are worth noting even though we observed no such use:

1. **The per-device gate is a targeting primitive.** A server that decides activation from the device's model and OS could in principle enable the proxy only on a chosen subset of devices (by model, or by the region it infers from the source IP), rather than all of them. The same mechanism that rations proxy load could single out a population.
2. **A tunnel restricted only against private ranges is a general-purpose "connect anywhere" capability.** Reselling exit bandwidth is the benign reading; the same primitive can reach any public host the operator names.

SECTION 1

Background & scope

The sample surfaced during a static sweep for proxyware SDKs across Android file-manager builds. The `io.nn.*` namespace and a `libnativesdk.so` with a Popa-style configuration pointed at the Popa family documented by Synthient, Qurium and XLab. (It also fits the broader SDK-delivered residential-proxy pattern HUMAN Security calls PROXYLIB, although PROXYLIB is a separate operation.)

Scope. Static analysis only, of a single build, **X-Plore 4.43.10**. Ghidra over the native library `libnativesdk.so` (arm64-v8a), and `jadx` over the app's Java, recovered from the **same build's** DEX after removing the pairip wrapper that encrypts it in the shipped APK (see Detection). The Java evidence (`io.nn.alpha` classes, `T2/*`, `p053e8/*`) and the native `.so` therefore come from one build, 4.43.10. Cross-referenced with passive DNS, certificate transparency, RDAP and VirusTotal history. No dynamic execution; the host app is integrity-protected (Limitations).

Analysis target: `libnativesdk.so` sha256 `0792897cec2a43eaa1f806e8e477f11aab73cccb6c91a8e03504ea78fd76f0ba`, NDK r25b / clang 14.0.6, stripped, BuildID `bddb8f4e2d49ffdd385e561ca5c8f6f91b3ba0a3`, build path `/opt/jenkins/workspace/gms_alpha/alphapop/src/main/cpp/mbed/`.

SECTION 2

Technical analysis

Family identification and placement. The package is `io.nn.alpha`, with Java classes `AlphaConfig`, `AlphaPop`, and `sdk_actions/AlphaSdkEngine`. [confirmed] Synthient's family table (Figure: Moneytiser `io.moneytise`, 2020-12; Popa `io.popanet`, 2022-03; Loopop `io.nn.lp`, 2023-11; Neupop `io.nn.neunative` / `io.nn.nativesdk`, C++, earliest 2026-02) does **not** list an "Alpha" entry, though Qurium and XLab name the Alpha variant (`libalphasdk`) and the relay domains separately. Our sample is the **same C++ native generation as Synthient's "Neupop"** (a `libnativesdk.so`, not the earlier Java builds), at the leading edge of the family's move to native code. [confirmed, from the local build archive] We extracted `libnativesdk.so` from each local X-Plore build and read its JNI exports:

`Java_io_nn_alpha_*` in 4.40.01 and 4.43.10 (three distinct BuildIDs across versions), and `Java_io_nn_neunative_*` in 4.43.14, which is where the rename happens. The package itself first appears one build earlier, in 4.39.02. Release dates are not in the local APKs (their zip timestamps are zeroed), so we take them from APKMirror: 4.39.02 uploaded 2024-09-25, 4.40.01 uploaded 2024-10-05. [high-confidence] That dates the Alpha-labelled native generation to **at least October 2024**, roughly 16 months before the earliest-seen sample Synthient assigns the native generation (2026-02); the two dates are not the same basis (ours is an APK build/upload date, Synthient's an earliest-seen date), so we present it as a lower bound, not an exact lead.

[confirmed] The clearest lineage tell is an incomplete rebrand: the module binds a system-wide `LocalServerSocket` named **`NEUPOP_LOCK`** (`p053e8/f.java:25`, bound at `e.java:29`), while every log tag in the same code path reads **`AlphaSocketLock`**. The cosmetic strings were renamed to "Alpha"; the functional lock constant was not. [high-confidence] This shows a shared codebase with the NEUPOP/Neupop line; it does not by itself fix the rename direction. [high-confidence] A sweep of 169 builds across 28 other packages (matching on the native-library and family signature, not a DEX string, since pairip encrypts the DEX) found this SDK in none of them; X-Plore is the only carrier we observed in that set; Synthient's publisher table also lists it, as `xp_gms`.

Two-stage bootstrap and the device gate. [confirmed] Before the proxy runs, `T2/d.java` + `T2/a.java` perform a device-compatibility check: an HTTPS GET to `https://lb.<domain>/initreq/?os=android&ver=<sdkver>&manufactor=<mfr>&model=<model>`, per domain, 3s timeout, 5 retries, then the next domain. The SharedPreferences flag `init_allowed` is set true **only if the response body contains the substring `YES`**; otherwise the SDK refuses to start (`p011b/d.java`, the `init_allowed` check gates the start path). [confirmed] the client only starts on `YES`. [unknown] whether the server ever returns anything else, or selects by device; that is server-side behaviour we cannot observe.

Configuration scheme (shared with Popa). [confirmed] The domain list has a hardcoded fallback (`p053e8.c.f50713a`) and a refreshable source. The refresh (`DomainsProvider`, native; and `T2/a.b()`, Java) fetches an encrypted blob, decrypts it with **AES-128 / key `eeorahrabcap286!`**, and splits the plaintext on commas. The native side's config object is the Google Drive URL `hxxps://drive[.]google[.]com/uc?export=download&id=1K95AXo75gi-jJSE9vuVPVEyBya0JUm0w`. Synthient's published Popa IOCs list the **same Drive ID and the same key**, decrypting (per Synthient) to `gmslb[.]net, fast-mob[.]com, sdkmob[.]org, swift-zip[.]com`; we did not fetch or decrypt the object ourselves. [confirmed] Java and native share the key and format.

Registration and what leaves the device. [confirmed] After the load balancer returns peer servers, the SDK opens a TLS connection to a control server and sends a Register message in a custom TLV wire protocol (4-byte type + 4-byte length header, big-endian integers). The fields:

TLV	Field	TLV	Field
0x6050	uuid (random v4, persisted)	0x6058	wifi flag
0x6051	dev_country	0x6059	model
0x6052	dev_city	0x605a	net_type
0x6053	publisher	0x605b	sdk_int
0x6054	sdk_version	0x605c	{"extra":..,"packageName":"<host app>"}
0x6055	dev_asn	0x605d	(registered, never set)
0x6056	carrier	0x605e	constant 1
0x6057	dev_state		

The geo fields (country/state/city/asn) are supplied by the load balancer's JSON reply and echoed back; the load balancer geolocates the device from the connection and the phone reports it onward. The **host application's package name travels in TLV 0x605c** on every registration, confirmed from Java (AlphaSdkEngine.startSdk1 passes getPackageName()) through to the native serializer.

The publisher ID (0x6053) is set by the host app, and for X-Plore it is xp_gms. [confirmed] X-Plore's own code builds the config with a hardcoded value: com/lonelycatgames/Xplore/l.java calls alphaConfig.setPublisher("xp_gms"), and the surrounding method logs Init Divi, X-Plore's internal name for the module. That the host app chose and hardcoded its own publisher ID is evidence of a deliberate, identified integration (it does not by itself establish whether the integration was witting or paid). The same mapping appears in Synthient's published publisher table, which lists xp_gms against com.lonelycatgames.Xplore alongside 25 other publisher IDs, most of them in the same <app>_gms form; what this report adds is reading it directly out of X-Plore's own code.

Command set. [confirmed] The message decoder accepts exactly seven server-to-client types; anything else logs Got invalid message and disconnects:

Type	Message	Action
0x5060	Register	no-op
0x7070	RegisterResponse	mark registered
0x9010	Ping	reply Pong
0xa070	OpenTunnel(id, host, port)	open TCP tunnel
0xc000	Tunnel data	forward to tunnel
0xcccc	CloseTunnel(id)	close tunnel
0xdddd	Goodbye	Engine::stop (remote kill)

The tunnel. [confirmed] OpenTunnel carries three TLVs: tunnel id (ulong), host (char*), port (int). The path is TunnelManager::openTunnel -> new Tunnel -> NetworkHandler::connect -> DnsResolver::resolveHost -> socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK) + connect(). The phone connects wherever the server names, IPv4 TCP only. The **one destination restriction** is in DnsResolver::processRequest: it takes the first AF_INET result and fails the lookup

if `Utils::isLocalAddress()` is true, covering `10/8, 172.16/12, 127/8, 192.168/16, 169.254/16, 224/3`. [confirmed] Not covered: `0/8, 100.64/10` (CGNAT), `198.18/15, 192.0.0/24`. No port restriction anywhere. Tunnel payloads are plain TCP (not TLS); the control channel is TLS.

TLS posture. [confirmed] mbedTLS 3.4.0, statically linked. The certificate chain is validated against the system CA store, but `mbedtls_ssl_set_hostname` has **no callers**, so there is no hostname/SAN check on the load balancer, control server or Drive connections. Any certificate chaining to a system-trusted CA is accepted.

Resilience and stealth. [confirmed] A `BOOT_COMPLETED` receiver and `START_REDELIVER_INTENT` keep the foreground service (`dataSync` type) alive; on API >= 34 a caught `ForegroundServiceStartNotAllowedException` defers the start until the host app is foregrounded. A crash handler (`sigaction` on SEGV/ABRT/BUS) reports the crash to the control server as a StatusUpdate (TLV `0x1112`: uptime, signal, publisher, version, OS, device, backtrace) and then `siglongjmp`s back into the worker loop rather than dying. A 60-second bandwidth meter stops the SDK once usage exceeds a server-set cap.

SECTION 3

Infrastructure & pivots

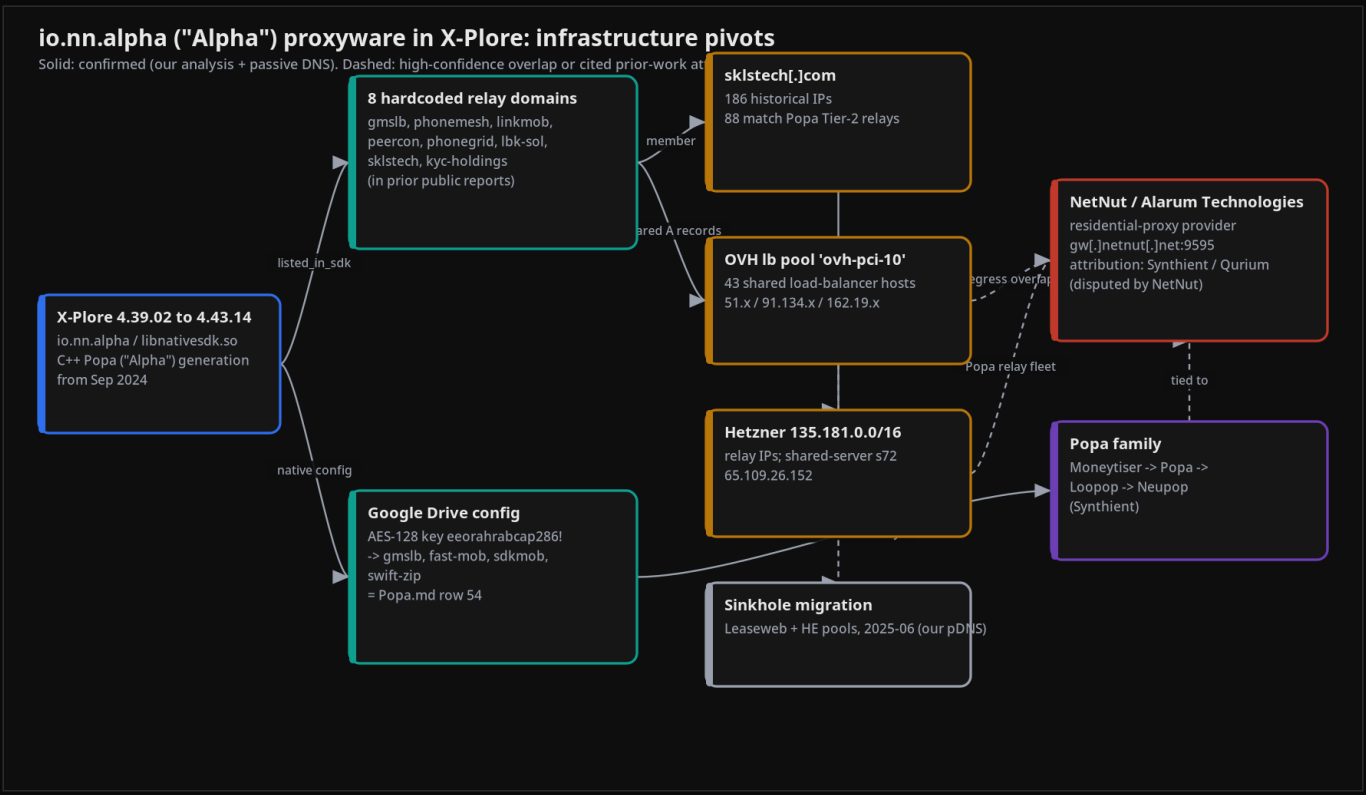


Figure 1. Pivot graph. Solid edges are confirmed from our static analysis and passive DNS; dashed edges are high-confidence infrastructure overlap or attribution cited from prior work (Synthient, Qurium). SVG and PNG in this issue's folder.

Eight domains are hardcoded in the SDK (`p053e8.c.f50713a`); three more come from the Popa Drive config.

Indicator	Pivot type	Linked to	Confidence	Evidence
`sklstech[.]com`	DNS / relay pool	Popa Tier-2 relays	high	186 historical IPs; 88 match Synthient Popa Tier-2
`lb.<domain>` pool	shared A records	43-IP OVH LB pool	high	`ovh-pci-10` pool shared across 43 tracked hosts
`65.109.26[.]152`	shared server	`sklstech[.]com` + a byte-buff host	high	same reverse-DNS server number (s72), Hetzner
`gmslb[.]net`	Drive config	Synthient's published Popa IOCs	confirmed	same Drive ID + AES key as Synthient's Popa data
OVH + Hetzner relays	ASN / hosting	Popa fleet	low	shared hosting ranges (`135.181.0.0/16`, OVH PCI); weak on its own
sinkhole migration	re-pointing	Popa cluster	hypothesis	relays re-pointed into Leaseweb + HE ranges by 2025-06 (our passive DNS, provider: RobTex/VirusTotal)

SECTION 4

Comparison to prior campaigns, and what is new here

The family is Popa, documented by Synthient ("Popa: From Sourcing to Distribution," June 2026), Qurium ("Finding Popa"), and XLab, and it fits HUMAN Security's broader PROXYLIB pattern of SDK-delivered residential proxyware. Synthient ties the family (Moneytiser, Popa, Loopop, Neupop) to the residential-proxy provider **NetNut**, whose parent **Alarum Technologies Ltd** is publicly traded; a controlled Synthient test on 2026-06-17 observed traffic from a Popa-enrolled host egressing through NetNut's commercial gateway (`gw[.]netnut[.]net:9595`, path `/NETNUT_EXT_TRAFFIC_FROM_PROXY`). Qurium links the **NinjaTech** platform (`ninjatech[.]io`, registered 2020-01-29) to NetNut leadership from public business records, and XLab connects the family to the **Vo1d** Android-TV botnet. NetNut/Alarum dispute these characterizations. In July 2026 the FBI, with Shadowserver, Google and Lumen, seized hundreds of domains associated with NetNut. None of this attribution or takedown is our work; it is cited.

What this report corroborates: our binary independently shows the same *architecture* Synthient describes, a load balancer that hands back a relay list, with `s0`/`s`-numbered relays on port 6000, reached before the control channel. The specific endpoints differ by config, which is expected across publishers: our build issues `GET /regdev?usr=...` to `lb.<domain>:443` (confirmed from the binary), whereas Synthient documents `/devicereg:5002` and `sXXXX.byte-buff[.]com:6000` relays in the samples they analysed. Same mechanism, different publisher config. The AES Google-Drive config scheme is likewise shared with Synthient's Popa documentation (see below).

What is new here: 1. **The affected build range.** Synthient's publisher table lists X-Plore (`xp_gms`) as a Popa carrier but gives no dates or versions. We bracket it: the package first appears in 4.39.02, the Alpha native library is present from 4.40.01, the exports are renamed at 4.43.14, and the module is gone by 4.44.00. 2. **An earlier native-generation sample.** The Alpha native library is in X-Plore from October 2024 (package from September 2024), a lower bound roughly 16 months before the earliest-seen sample Synthient dates to this native generation (2026-02). It pushes the family's native-code timeline back and shows the `NEUPOP_LOCK`-under-"Alpha" state directly in a dated, retrievable build. 3. **Native-protocol depth**, extending Qurium's description of the "Popa TLV" tunnelling protocol: the full seven-type command set including the remote `Goodbye` kill, the `OpenTunnel` connect-anywhere path with its single private-range filter, the complete Register TLV map, and the `mbedtls_ssl_set_hostname`-has-no-callers finding, all read from the binary. 4. **Tested detection** keyed on the native library and network behaviour, with the pairip-encrypted-DEX caveat spelled out.

Our Drive object and AES key match Synthient's Popa dataset rather than differing from it, so that is a corroboration of the family link, not a new finding; a different key (`venlutDakros643!`) appears in the sample in Synthient's blog, so distinct publisher configs exist, but we do not claim a specific branch from one differing value.

Version timeline. [confirmed, from local builds + APKMirror] The `io.nn.alpha` package first appears in X-Plore 4.39.02 (uploaded 2024-09-25); the prior build 4.38.40 is clean. The Alpha native library (`Java_io_nn_alpha_*` exports) is confirmed in 4.40.01 (2024-10-05) and in our 4.43.10 sample; at 4.43.14 the native exports become `Java_io_nn_neunative_*`, which is the rename. The module is gone by 4.44.00. Total presence roughly nine months, each build carrying a distinct `.so` BuildID. [caveat] AndroZoo and these local builds are not a complete release history, and a rebuilt or re-labelled variant could evade a signature sweep.

SECTION 5

IOC reference

Type	Indicator	Note
domain	`gmslb[.]net`	SDK; also in Synthient's Popa Drive config
domain	`phonemesh[.]org`	SDK hardcoded
domain	`linkmob[.]org`	SDK hardcoded
domain	`peercon[.]org`	SDK hardcoded
domain	`phonegrid[.]org`	SDK hardcoded
domain	`lbk-sol[.]com`	SDK hardcoded
domain	`sklstech[.]com`	SDK hardcoded; 186-IP relay history
domain	`kyc-holdings[.]com`	SDK hardcoded
domain	`fast-mob[.]com`	Popa Drive config (candidate)
domain	`sdkmob[.]org`	Popa Drive config (candidate)
domain	`swift-zip[.]com`	Popa Drive config (candidate)
url	`hxxps://drive[.]google[.]com/uc?export=download&id=1K95AXo75gi-jJSE9vuVPVEyBya0JUm0w`	encrypted domain config
key	`eeorahrabcap286!`	AES-128 config key (shared with Popa)
sha256	`0792897cec2a43eaa1f806e8e477f11aab73cccb6c91a8e03504ea78fd76f0ba`	`libnativesdk.so` (arm64, X-Plore 4.43.10)
build id	`bddb8f4e2d49ffdd385e561ca5c8f6f91b3ba0a3`	native library BuildID
package	`io.nn.alpha`	SDK package namespace
socket	`NEUPOP_LOCK`	LocalServerSocket name (lineage tell)
publisher	`xp_gms`	X-Plore's publisher ID (Register TLV `0x6053`); also listed in Synthient's Popa publisher table

SECTION 6

Detection

Three rules, in `detections/`. The native YARA and the Sigma are tested as described below; the DEX rule only compiles, because it cannot run against the shipped (pairip-encrypted) APK. "Tested" here means compiles and matches the one sample with no hits on a small clean corpus; it is not a false-positive rate against a large representative set.

Where detection is and is not possible. The shipped APK is pairip-wrapped: its real DEX, the `io.nn.alpha` classes included, lives in encrypted asset blobs (randomly-named ~120KB files under `assets/`) that the native VM decrypts at runtime. So `io.nn.alpha`, `NEUPOP_LOCK` and the `/initreq` template are **not present as plaintext in the distributed APK**, and DEX string-scanning will miss them. Reliable detection keys on the native library (plaintext, loaded normally from `lib/`) and on network telemetry. A DEX rule is provided for analysts working from a decrypted or dumped DEX.

1. Native library (primary file signature). Tested: compiles, matches the sample (1/1), zero hits on a small clean corpus (system binaries, the x86_64 `frida-server`, and the proxy-free X-Plore 4.44.00).

```
rule proxyware_io_nn_alpha_native
{
    meta:
        description = "io.nn.alpha residential-proxy native SDK (libnativesdk.so), Popa/Neupop
lineage"
        author = "Bornov Shyam Kalita"
        date = "2026-10-16"
        reference = "GhostWire Issue 003 (2026-BSK-003)"
        hash = "0792897cec2a43eaa1f806e8e477f11aab73cccb6c91a8e03504ea78fd76f0ba"
        confidence = "high-confidence: build path and JNI symbol are alpha-specific; the AES key is
shared across the Popa family so it broadens to the family, not just this variant"
    strings:
        $build = "/opt/jenkins/workspace/gms_alpha/alphapop/src/main/cpp/mbed/" ascii
        $jni   = "Java_io_nn_alpha_sdk_1actions_AlphaSdkEngine_" ascii
        $key   = "eeorahrabcap286!" ascii
        $c1 = "LBConnectionHandler" ascii
        $c2 = "DomainsProvider" ascii
        $c3 = "Alpha_SDK" ascii
        $c4 = "SdkProtocolOpenTunnelMessage" ascii
    condition:
        uint32(0) == 0x464c457f and filesize < 8MB and
        ( $build or $jni or ( $key and 2 of ($c*) ) )
}
```

2. Network beacon (packing-agnostic, runtime). Sigma (parses against the current schema, `status: experimental`) keying on the per-device `/initreq` gate. Condition `selection_uri and (selection_ua or selection_host)`: a GET to `/initreq` carrying `os=android` + `manufactor=` +

`model=`, with User-Agent `SDK` (that UA string is hardcoded in the binary, `buildResponseStr`) or a `lb.` host prefix. Caveat: the URI and User-Agent fields are only visible to a sensor doing TLS interception, since the beacon is HTTPS; without that, rely on the DNS companion rule that alerts on resolution of the eight hardcoded relay domains (see IOC table).

3. Decrypted DEX only (scoped). Compiles; **not** testable against the shipped APK because the strings are pairip-encrypted at rest. `detections/alpha_dex_decrypted.yar` keys on `io/nn/alpha`, the `AlphaService` class, `NEUPOP_LOCK`, and the `/initreq` template, for a decrypted or dumped DEX only. A live-device companion: the SDK binds a `LocalServerSocket` named `NEUPOP_LOCK`, visible in `/proc/net/unix`, which survives pairip.

SECTION 7

ATT&CK mapping

MITRE ATT&CK for Mobile, with the one enterprise technique that captures the proxy behaviour itself. Each row maps to evidence documented above.

Tactic	Technique	ID	Evidence
Persistence	Boot or Logon Initialization Scripts	T1398	`BootReceiver` on `BOOT_COMPLETED` restarts the SDK
Persistence	Foreground Persistence	T1541	`dataSync` foreground service, `START_REDELIVER_INTENT`
Defense Evasion	Masquerading	T1655	proxy module embedded in a legitimate file manager
Defense Evasion	Execution Guardrails	T1627	`/initreq` device gate: starts only on a server `YES`
Defense Evasion	Obfuscated Files or Information	T1406	pairip-encrypted DEX; AES-128 Google-Drive config
Command and Control	Application Layer Protocol	T1437.001	HTTPS to the load balancer (`/regdev`, `/initreq`)
Command and Control	Encrypted Channel	T1521.001	AES-128 config; mbedTLS TLS control channel
Discovery	System Information Discovery	T1426	model, SDK level, carrier, network type in the Register message
Impact (Enterprise)	Proxy	T1090	the device relays third-party traffic as a residential exit node

SECTION 8

Limitations

- **No dynamic validation.** This is static analysis. The host app (X-Plore 4.43.10) is wrapped with Google Play Automatic Integrity Protection (pairip): the real DEX runs through a native VM decrypted into anonymous memory, plus an APK signature check. On

an x86_64 emulator with arm64 translation the process SIGSEGVs at launch, and current Android emulators no longer emulate arm64 on x86 hosts. Confirming the connect-anywhere tunnel behavior live, observing real control-server traffic, and recovering the runtime-decrypted init URL all require a **native arm64 instrumentation platform (for example Corellium) or a dedicated rooted arm64 device**, plus handling of the pairip layer. This is the single largest gap and the natural next phase.

- **Server-side behavior is unknown.** The client capabilities are confirmed from the binary; what the operator actually does with them (which destinations, which devices selected by the `/initreq` gate) cannot be determined from the client alone.
- **One inferred direction.** `TunnelManager::sendToTunnel` (server-to-destination path) is named but its body was not fully traced; the direction is inferred from the symbol. [high-confidence, not confirmed]
- **Config contents not independently decrypted.** The Drive config plaintext (`gmslb[.]net, fast-mob[.]com, sdkmob[.]org, swift-zip[.]com`) is per Synthient; it was not fetched or decrypted here by design (the live object was deliberately not pulled).
- **Obfuscated init URL.** The Java config-fetch URL is runtime-decrypted via a string-obfuscation shell (`p077h.LOj`) and was not recoverable from `jadx` alone.
- **Attribution is prior work's, not ours.** The link from this family to NetNut / Alarum Technologies / the NinjaTech platform is established by Synthient and Qurium from network telemetry and public business records, and is disputed by NetNut. This report does not independently verify that linkage. What a planned addendum (Issue 003a) would add is tying X-Plore's specific relay cluster to that documented operation, not fresh corporate attribution.
- **Novelty scope.** "No public report names X-Plore as a carrier" rests on Qurium, Synthient and open search; I could not read XLab's original disclosure in full, so there is a residual chance the carrier is named in a source not reviewed here.

SECTION 9

References

1. Synthient, "Popa: From Sourcing to Distribution," June 2026. https://synthient.com/blog/popa-from-sourcing-to-distribution (family table, the `/initreq` gate, the NetNut/Alarum attribution and controlled egress test).
2. Qurium Media Foundation, "Finding Popa." https://www.qurium.org/forensics/finding-popa/ (names the Alpha variant / `libalphasdk` and the relay domains; NinjaTech-to-NetNut business-record linkage).
3. HUMAN Security (Satori Threat Intelligence), "PROXYLIB and LumiApps transform mobile devices into proxy nodes," March 2024. https://www.humansecurity.com/learn/blog/satori-threat-intelligence-alert-proxylib-and-lumiapps-transform-mobile-devices-into-proxy-nodes
4. AndroZoo, University of Luxembourg. https://androzoo.uni.lu (build archive used for the version bracketing).
5. KrebsOnSecurity, "'Popa' Botnet Linked to Publicly-Traded Israeli Firm" (June 2026) and "FBI Seizes NetNut Proxy Platform, Popa Botnet" (July 2026). https://

krebsonsecurity.com/tag/popa-botnet/ ; https://krebsonsecurity.com/2026/07/fbi-seizes-netnut-proxy-platform-popa-botnet/

6. XLab (QiAnXin), "Long Live The Vo1d Botnet," identifying the Mzmess/Popa proxy plugin inside the Vo1d Android-TV botnet. https://blog.xlab.qianxin.com/long-live-the-vo1d_botnet/
7. FBI / Shadowserver / Google / Lumen seizure of the NetNut network and the Popa cluster, July 2026 (Shadowserver assisted with sinkholing and notification). https://www.infosecurity-magazine.com/news/fbi-google-take-down-netnut-proxy/ . Separately, our own passive-DNS pull shows the Alpha relay fleet migrating to Leaseweb/HE sinkhole pools by June 2025.